



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Carrera:

Licenciatura en Ciencia de Datos

Unidad de Aprendizaje:

Modelado Predictivo

Profesor:

Sánchez Díaz Hugo

---- PROYECTO CLASIFICACIÓN ----

Clasificación Automática De Residuos Con Aprendizaje Profundo Para Apoyar El Reciclaje

Integrantes:

Cortes Rosales Israel

Dorado Alcalá Teresa Nathaly

Gallegos Sánchez Celso Alberto

Monteros López José Manuel

Pacheco Molina Miguel Alejandro

Palma Pacheco Alan Daniel

Grupo:

6AV1

15 de diciembre de 2025



ÍNDICE GENERAL

ÍNDICE DE TABLAS Y FIGURAS	3
RESUMEN.....	4
INTRODUCCIÓN	5
OBJETIVOS	5
JUSTIFICACIÓN	6
DESCRIPCIÓN DE LOS DATOS	6
MARCO TEÓRICO / REVISIÓN DE LITERATURA	7
PREPARACIÓN DE DATOS	18
METODOLOGÍA.....	19
1. Muestreo (Sample)	19
2. Exploración (Explore)	19
3. Modificación (Modify).....	21
4. Modelado (Model).....	21
5. Evaluación (Assess).....	25
RESULTADOS	26
DISCUSIÓN.....	26
CONCLUSIONES	27
REFERENCIAS	27

ÍNDICE DE TABLAS Y FIGURAS

Figura 1: Ejemplo del flujo de procesamiento en una red neuronal convolucional.....	9
Figura 2: Esquema conceptual de Transfer Learning	10
Figura 3: Ejemplo del Muestras de Data Augmentation	11
Figura 4: Comparación conceptual	12
Figura 5: Estructura de un bloque residual.....	13
Figura 6: Fórmula para calcular la actualización de pesos del optimizador Adam	14
Figura 7: Registro del entrenamiento	15
Figura 8: Formula para obtener el valor de Accuracy o exactitud.....	16
Figura 9: Formula para obtener el valor de Precision	16
Figura 10: Formula para obtener el valor de Recall.....	17
Figura 11: Formula para obtener el valor de f1-score	17
Figura 12: Visualización de una matriz de confusión.....	18
Figura 13: Distribución de imágenes por clase	19
Figura 14: Ejemplos de clase.	20
Figura 15: Matriz de confusión – Baseline CNN.	22
Figura 16: Matriz de confusión – ResNet18.	23
Figura 17: Matriz de confusión – ResNet18 (class_weight + augmentation).....	23
Figura 18: Matriz de confusión – ResNet50 Congelada.	24
Figura 19: Curva ROC Multiclase ResNet50 Congelado.	25
Figura 20: Curva Precision - Recall Multiclase ResNet50 Congelado.....	25

RESUMEN

Este proyecto tiene como propósito desarrollar un sistema que clasifica imágenes de basura doméstica en doce categorías, con la intención de apoyar y optimizar los procesos de reciclaje. Para ello se empleó un conjunto de datos integrado por 15,150 imágenes que representan distintos tipos de residuos.

El estudio siguió un proceso ordenado que incluyó la revisión del conjunto de datos, la preparación de las imágenes y la construcción y evaluación de diferentes modelos. Se compararon tres alternativas: un modelo inicial sencillo, otro que aprovecha una red previamente entrenada y una tercera versión que incorpora técnicas para aumentar la variedad de imágenes y atender el desbalance entre categorías.

Los resultados muestran una mejora significativa en desempeño al aplicar técnicas avanzadas de transferencia y balanceo, demostrando que un enfoque robusto puede mejorar la clasificación automática de residuos.

Palabras clave: Clasificación de residuos; Imágenes; Inteligencia artificial; Modelos de aprendizaje; Reciclaje.

INTRODUCCIÓN

El manejo adecuado de los residuos es uno de los desafíos ambientales más relevantes en la actualidad. A pesar de los esfuerzos por promover prácticas de reciclaje, la clasificación manual de desechos continúa siendo un proceso lento, costoso y susceptible a errores humanos. Estas limitaciones dificultan la eficiencia de los sistemas de gestión ambiental y reducen el impacto positivo que podría alcanzarse mediante una separación adecuada.

En este contexto, las tecnologías de visión por computadora ofrecen una alternativa prometedora para automatizar la clasificación de residuos. Mediante el uso de modelos capaces de identificar patrones visuales, es posible distinguir materiales como papel, plástico, vidrio, metal o textil a partir de imágenes, lo que abre la puerta a soluciones más rápidas, precisas y escalables. En particular, las redes neuronales convolucionales han demostrado un desempeño sobresaliente en tareas de reconocimiento de imágenes, lo que las convierte en una opción viable para este tipo de aplicaciones.

En este proyecto se estudia el uso de modelos de clasificación de imágenes para identificar distintos tipos de basura doméstica, tales como papel, plástico, vidrio o metal. Para ello, se utiliza un conjunto diverso de fotografías de residuos reciclables y se comparan varias estrategias de modelado con el fin de identificar aquella que ofrezca mejores resultados. Con este análisis, se busca aportar una solución técnica que pueda contribuir a mejorar la eficiencia de los procesos de reciclaje y fortalecer las prácticas de gestión ambiental.

OBJETIVOS

Objetivo general

Desarrollar y evaluar un sistema automático de clasificación de imágenes que identifique el tipo de residuo presente en una fotografía, asignándolo correctamente a una de las doce categorías definidas en el conjunto de datos, mediante la aplicación de técnicas de aprendizaje profundo y métodos de evaluación estandarizados.

Objetivos específicos

- Analizar y preparar los datos mediante la exploración del conjunto de imágenes, la estandarización de sus dimensiones y la aplicación de técnicas de preprocesamiento adecuadas para su uso en modelos de aprendizaje profundo.
- Implementar un modelo base utilizando una arquitectura sencilla de redes neuronales convolucionales (CNN), con el fin de establecer un punto de comparación para modelos más avanzados.
- Aplicar técnicas de *Transfer Learning* empleando modelos preentrenados en grandes bases de datos (como ImageNet) para comparar su desempeño frente al modelo base y determinar mejoras en exactitud y eficiencia.

- Incorporar estrategias de aumento de datos y manejo del desbalance mediante técnicas como rotaciones, escalado y ponderación de clases para mejorar la capacidad de generalización de los modelos.
- Evaluar el rendimiento de los modelos mediante métricas estandarizadas y determinar la estrategia más eficiente para la clasificación de residuos.

JUSTIFICACIÓN

La correcta separación de residuos es un paso fundamental para mejorar los procesos de reciclaje y avanzar hacia prácticas más sostenibles de gestión ambiental. Sin embargo, la clasificación manual suele ser lenta, costosa y propensa a errores, lo que limita la capacidad de recuperación de materiales útiles.

La automatización basada en análisis de imágenes representa una alternativa viable para reducir los tiempos de operación, disminuir la dependencia de trabajo manual e incrementar la precisión en la identificación de materiales reciclables.

Además, la mayoría de los conjuntos de datos disponibles para este tipo de tareas incluyen pocas categorías, generalmente entre dos y seis clases. Este proyecto se distingue por trabajar con un conjunto de doce categorías, lo que permite una clasificación más detallada y cercana a las necesidades reales de los sistemas de reciclaje. Al evaluar diferentes enfoques de modelado, el estudio aporta evidencia útil para el diseño de soluciones tecnológicas que fortalezcan los esfuerzos de sostenibilidad y mejoren la recuperación de materiales aprovechables.

DESCRIPCIÓN DE LOS DATOS

El conjunto de datos utilizado en este estudio corresponde al “Garbage Classification (12 classes)”, disponible en la plataforma Kaggle y compilado por Mostafa Abla. Este dataset está conformado por 15,150 imágenes de residuos domésticos distribuidas en doce categorías: papel, cartón, orgánico, metal, plástico, vidrio verde, vidrio café, vidrio blanco, ropa, zapatos, baterías y basura general.

El conjunto fue elaborado a partir de imágenes recolectadas mediante web scraping, con el fin de representar condiciones reales de disposición de residuos. Como resultado, las fotografías presentan una amplia variedad de fondos, niveles de iluminación, ángulos de captura y estados físicos de los objetos, lo que incrementa el realismo del dataset y la complejidad del problema de clasificación.

Para facilitar el manejo de los datos, se construyó un DataFrame estructurado que organiza las rutas de las imágenes junto con sus etiquetas correspondientes. Además, se realizaron visualizaciones y muestreos por clase que permitieron observar ejemplos representativos y comprender la distribución y naturaleza del conjunto de imágenes.

MARCO TEÓRICO / REVISIÓN DE LITERATURA

En el presente apartado se establecen los fundamentos conceptuales y técnicos necesarios para comprender el uso de visión por computadora y aprendizaje profundo en la clasificación automática de residuos.

1. Visión por Computadora y Gestión de Residuos

1.1. Contexto del Reconocimiento de Patrones

La visión por computadora es una rama de la inteligencia artificial cuyo objetivo es procesar, analizar y comprender imágenes del mundo real para producir información útil (Szeliski, 2010). A través de los años se ha convertido en una de las herramientas más poderosas para tareas de clasificación y reconocimiento de patrones (Goodfellow et al., 2016), sin embargo, su aplicación en la gestión de residuos presenta desafíos particulares que dificultan su automatización donde el principal problema es que los residuos de algún mismo tipo no siempre se parecen entre sí y a la vez residuos de tipos diferentes pueden parecerse mucho lo que complica que un modelo los distinga.

- **Deformabilidad:** A diferencia de objetos fabricados que suelen mantener la misma forma, los residuos como botellas de plástico, latas de metal, cartón entre otras son objetos no rígidos que pueden aplastarse, romperse, arrugarse o deformarse de muchas maneras antes de ser fotografiadas lo que hace que la apariencia de un mismo material cambie drásticamente entre imágenes.
- **Ambigüedad Visual:** Materiales distintos pueden verse muy similares como el vidrio transparente y ciertos plásticos u otros que comparten características visuales similares lo que complica que el modelo establezca fronteras de decisión claras, además de la calidad de imagen (iluminación, enfoque, ruido, etc..) también puede aumentar la confusión entre categorías.

1.2. Representación Matemática de la Imagen

Para que un algoritmo pueda procesar información visual, esta debe ser procesada. Formalmente una imagen digital se representa como la función de intensidad $I(x, y)$ muestreada sobre una rejilla discreta, en aprendizaje profundo, esta estructura se modela como un tensor tridimensional:

$$X \in \mathbb{R}^{H \times W \times C}$$

Donde:

- H (Height) y W (Width) representan el alto y ancho de la imagen medidas en píxeles.
- C (Channels) representa la profundidad espectral (cuantas capas de color tiene cada píxel), típicamente $C = 3$ para el espacio de color RGB(rojo, verde, azul).

Cada elemento de este tensor se denomina píxel, este almacena un valor numérico en el intervalo $[0, 255]$ que indica la intensidad lumínica en esa coordenada específica.

El principal reto para un modelo de clasificación es la *brecha semántica*: la computadora solo ve números organizados en una matriz, mientras que los humanos interpretamos esos números como objetos con significado (como “metal”, “vidrio” o “plástico”). El modelo debe aprender a transformar esa información numérica de bajo nivel en una decisión de alto nivel que indique a qué clase pertenece la imagen.

2. Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN o ConvNets) son el estándar actual en la visión por computadora. A diferencia de las redes densas tradicionales (*Perceptrones Multicapa*) que convierten la imagen en un vector y pierden la estructura espacial, las CNN están diseñadas para trabajar con datos en forma de rejilla, manteniendo las relaciones entre píxeles (LeCun et al., 1998; Goodfellow et al., 2016).

2.1 Mecanismo de funcionamiento de una CNN

Una CNN transforma la imagen de entrada en representaciones cada vez más abstractas mediante una arquitectura basada en capas. Sus dos operaciones principales son:

- **Convolución:** Es la operación central de la red la cual consiste en deslizar filtros pequeños (*kernels*) sobre la imagen. En cada posición se calcula un producto punto entre el filo y la región local, generando un mapa de características. Matemáticamente para una entrada I y un filtro K la operación se define como:

$$S(i, j) = (I \cdot K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

Esta operación permite detectar patrones locales como bordes o texturas sin importar dónde aparezcan en la imagen.

- **Pooling:** Después de las capas de convolución, las CNN suelen aplicar capas de pooling donde su función es reducir el tamaño de la imagen procesada de modo que la red use menos memoria y sea más rápida, la técnica más común es *Max Pooling*, que consiste en tomar una pequeña ventana (ej. 2x2 píxeles) y quedarse solo con el valor más alto dentro de ella, esto ayuda a:
 - Reducir la cantidad de información, manteniendo solo lo más importante.
 - Hacer la red más robusta si el objeto se mueve un poco o está ligeramente deformado.
 - Evitar que la red dependa de la posición exacta del objeto, porque pequeñas variaciones no cambian el resultado del pooling.

Así el pooling actúa como un filtro que resume zonas pequeñas de la imagen y hace que la red sea más eficiente y estable.

2.2. Aprendizaje de Representaciones (Feature Learning)

Antes del Deep Learning, la visión por computadora dependía de características diseñadas manualmente (como *SIFT* o *HOG*). Este enfoque es limitado con objetos con alta variabilidad, como la basura doméstica. Las CNN aprenden automáticamente de las características necesarias directamente a partir de los datos:

- Capas iniciales: Detectan bordes, líneas y gradientes.
- Capas intermedias: Combinan patrones simples para reconocer partes del objeto.
- Capas profundas: Forman representaciones completas y semánticas (ej. la silueta de una botella o una lata).

Esta capacidad de aprendizaje jerárquico es lo que permite que las CNN clasifiquen materiales deformados o con variaciones significativas sin intervención humana en el diseño de filtros.

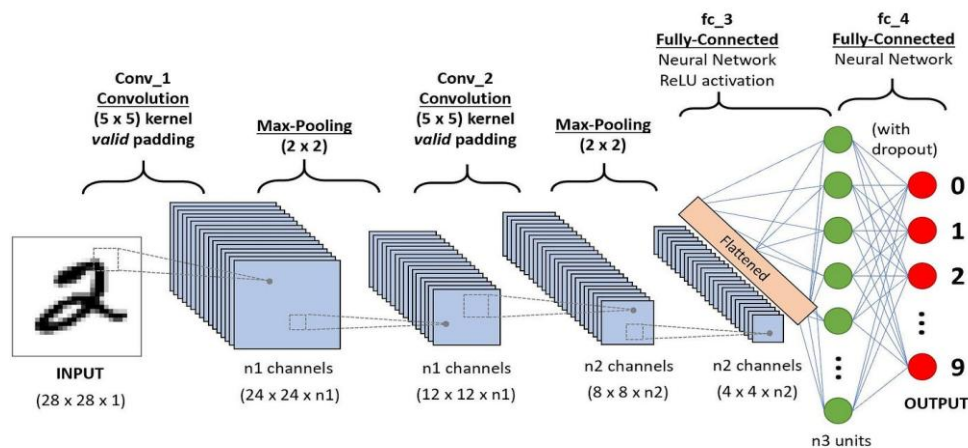


Figura 1: Ejemplo del flujo de procesamiento en una red neuronal convolucional.

3. Estrategias de Aprendizaje con Datos Limitados

Cuando se trabaja con modelos de *Deep Learning*, uno de los principales desafíos es la escasez de datos etiquetados, especialmente en problemas específicos como la clasificación de residuos. Las redes profundas pueden modelar patrones complejos, pero con pocos datos suelen sobreajustarse. Para reducir este riesgo se emplean 3 estrategias principales: *Transfer Learning* y *Data Augmentation* y *Análisis Discriminante*.

3.1. Transfer Learning (Transferencia de Aprendizaje)

Esta técnica consiste en aprovechar el conocimiento adquirido por un modelo entrenado previamente en una tarea grande por ejemplo *ImageNet* la cual cuenta con más de un millón de imágenes y luego reutilizarlo en una tarea nueva con pocos datos (Pan & Yang, 2010). Existen 2 formas de aplicarlo:

- **Feature Extraction:** Se congela la base convolucional del modelo pre-entrenado, utilizándola como un extractor de características fijo. Solo se entrenan los pesos de las nuevas capas densas (clasificador) añadidas al final. Esto reduce drásticamente el número de parámetros entrenables, disminuyendo el riesgo de sobreajuste.
- **Fine-Tuning:** Descongela algunas de las capas superiores de la base convolucional y se entrena junto con el clasificador utilizando una tasa de aprendizaje (*learning rate*) muy baja. Teóricamente, esto permite adaptar las representaciones abstractas de alto nivel a las particularidades visuales específicas del nuevo dominio (por ejemplo, diferenciar texturas de plásticos específicos frente a objetos genéricos).

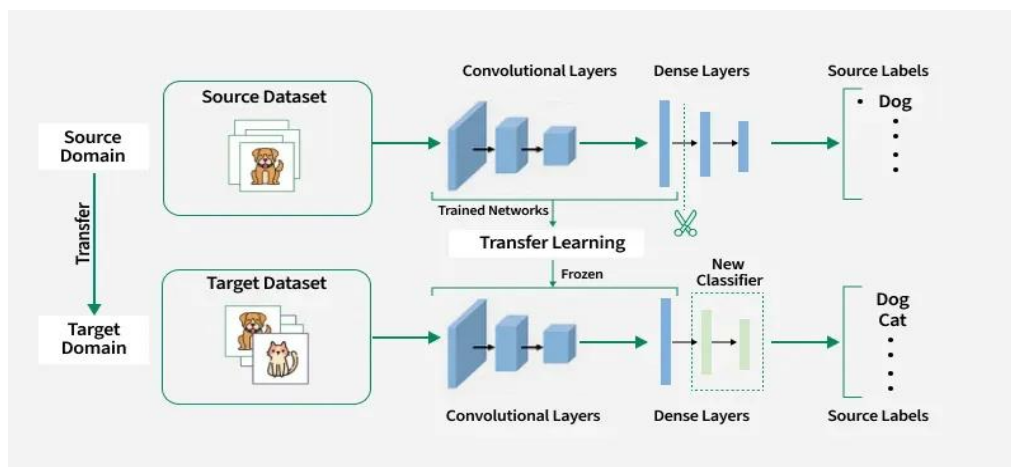


Figura 2: Esquema conceptual de Transfer Learning, las capas convolucionales entrenadas en un dominio masivo se transfieren y congelan, entrenando únicamente un nuevo clasificador para la tarea destino.

3.2. Manejo del Desbalance de Clases, Data Augmentation y Muestreo Ponderado

En los conjuntos de datos reales es común que haya muchas imágenes de unas clases y muy pocas de otras provocando que el modelo tienda a predecir siempre la clase mayoritaria afectando la capacidad de generalización.

Para tratar la falta de datos como el desbalance se utiliza el *Data Augmentation* que genera versiones modificadas de las imágenes originales aplicando transformaciones rotaciones, translaciones, cambios de escala(zoom) y modificaciones fotométricas (brillo/contraste).

Estas variaciones actúan como un regularizador ya que obligan al modelo a aprender patrones que sean robustos a cambios de ángulo, posición o iluminación, de esta manera la red aprende la esencia del objeto y no memoriza un ejemplo específico mejorando la generalización del clasificador.

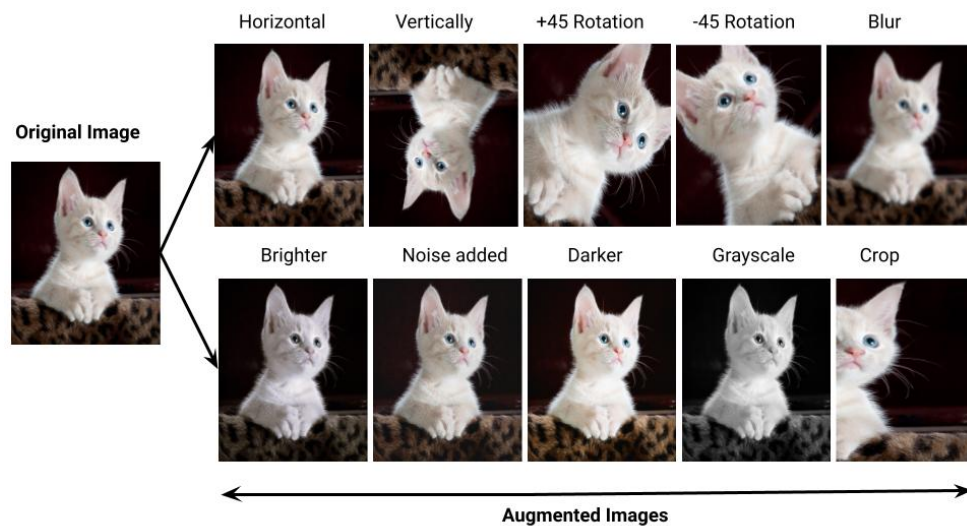


Figura 3: Ejemplo del Muestras de Data Augmentation aplicando transformaciones geométricas (rotación, recorte) y fotométricas (brillo, ruido) a una imagen original.

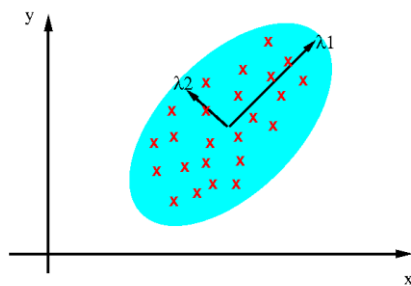
Para corregir el sesgo hacia la clase mayoritaria durante el entrenamiento, se emplea el *Muestreo Ponderado*. Esta técnica altera la probabilidad de selección de las muestras para formar los lotes (*batches*). Se asigna un peso inverso a la frecuencia de cada clase ($w_c = \frac{1}{N_c}$), forzando al algoritmo a procesar una proporción equilibrada de ejemplos de todas las categorías en cada iteración, independientemente de su abundancia original en el dataset.

3.3. Análisis Discriminante Lineal (LDA)

Aunque las CNN son excelentes extractores de características, las representaciones vectoriales de algunos materiales visualmente pueden solaparse en el espacio latente, generando confusión en el clasificador.

Para optimizar la distinción entre estas categorías, se integra el *Análisis Discriminante Lineal (LDA)*. A diferencia de métodos no supervisados (como *PCA*), LDA es una técnica supervisada cuyo objetivo matemático es encontrar una proyección que maximice la separabilidad entre clases (Fisher, 1936).

PCA: component axes that maximize the variance



LDA: maximizing the component axes for class-separation

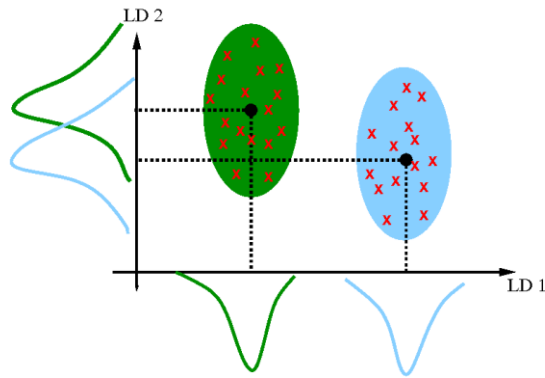


Figura 4: Comparación conceptual. PCA maximiza la varianza global (izquierda), mientras que LDA maximiza la separabilidad entre clases (derecha).

LDA optimiza el *Criterio de Fisher*, buscando maximizar la distancia entre las medias de las clases (S_B : dispersión inter-clase) minimizando simultáneamente la dispersión dentro de cada clase (S_W : dispersión intra-clase):

$$J(w) = \frac{w^T S_B w}{w^T S_W w}$$

Al aplicar LDA sobre las características extraídas por la red neuronal, se logra compactar los grupos de una misma clase y alejarlos de las otras, facilitando la decisión final del clasificador y reduciendo los falsos positivos en materiales críticos.

4. Arquitecturas de Deep Learning de Alto Desempeño

Para la clasificación, se seleccionaron arquitecturas que han redefinido el rendimiento en visión por computadora: *ResNet* por resolver la degradación en redes profundas y *EfficientNet* por su optimizar el uso de recursos sin perder precisión.

4.1. Redes Residuales (ResNet)

Una idea central en Deep Learning es que las redes más profundas aprenden características más complejas. Sin embargo, He et al. (2016) demostraron que aumentar la profundidad indiscriminadamente conduce al problema de *degradación del gradiente*, donde la precisión se satura y decae debido a la dificultad de optimización, no por sobreajuste.

Para solucionar esto, la arquitectura *ResNet* introduce el "bloque residual" con conexiones de salto (*skip connections*).

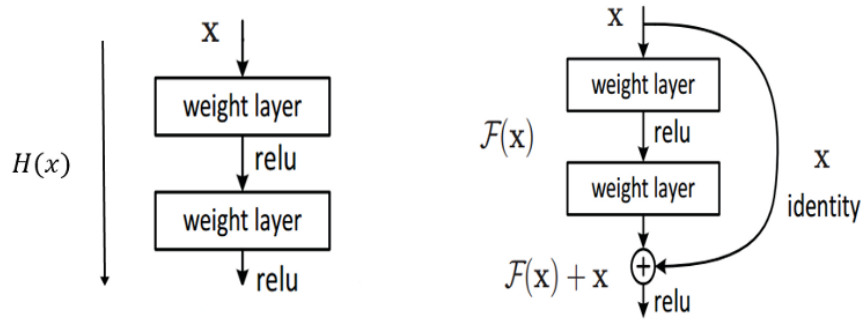


Figura 5: Estructura de un bloque residual. La conexión de salto (identity) permite sumar la entrada original a la salida de las capas de peso, facilitando el flujo del gradiente.

En lugar de intentar aprender un mapeo directo $H(x)$, las capas se reformulan para aprender una función residual:

$$F(x) = H(x) - x$$

Y la salida del bloque se define como:

$$H(x) = F(x) + x$$

Donde x es la identidad de la entrada. Esta formulación permite que, durante el entrenamiento, el gradiente pueda fluir directamente a través de las conexiones de salto sin atenuarse, actuando como ruta directa para la señal de error. Esto permitió el entrenamiento efectivo de redes muy profundas, como *ResNet50*, manteniendo una alta capacidad de generalización.

4.2. EfficientNet y Escalamiento Compuesto

Otro modelo que ha sido adoptado para diversas tareas de visión artificial, como la clasificación de imágenes, detección de objetos y segmentación es EfficientNet, una familia de redes neuronales convolucionales (CNN) para visión artificial desarrollada por Google. Su innovación clave es el escalado compuesto, que escala uniformemente y de forma simultánea las dimensiones de profundidad, anchura y resolución utilizando un único parámetro. (Tan, M., & Le, Q. V., 2019) Concretamente, dada una red de referencia, la profundidad, la anchura y la resolución se escalan según las siguientes ecuaciones:

$$\text{profundidad} = \alpha^\phi, \text{ amplitud} = \beta^\phi, \text{ resolución} = \gamma^\phi$$

donde α , β , γ son constantes determinadas mediante búsqueda en grid bajo restricción $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ y $\alpha \geq 1$, $\beta \geq 1$, $\gamma \geq 1$; y ϕ siendo el coeficiente compuesto para escalar las tres dimensiones simultáneamente.

Estas restricciones aseguran que duplicar recursos computacionales (medidos en FLOPs) duplica aproximadamente ϕ , manteniendo escalado predecible y eficiente. Los exponentes cuadráticos para β y γ reflejan que FLOPs de convoluciones crecen cuadráticamente con amplitud y resolución, pero linealmente con profundidad.

A diferencia de enfoques tradicionales que escalan arbitrariamente estas dimensiones, EfficientNet emplea búsqueda de arquitectura neuronal (NAS) para optimizar la estructura base y luego aplica escalado uniforme.

5. Optimización y Regularización

5.1. Algoritmos de Optimización Estocástica (Adam)

Adam (Adaptive Moment Estimation o Estimación Adaptativa de Momentos) es un algoritmo de optimización diseñado para actualizar los parámetros de una red neuronal durante el proceso de entrenamiento. Fue propuesto por D.P. Kingma y J.Ba en 2014 y combina las ventajas de otros dos métodos de optimización: el algoritmo de Gradiente Descendente Estocástico (SGD) y el optimizador RMSProp, de los cuales hereda la capacidad de adaptar dinámicamente el tamaño del paso para cada peso mientras mantiene una tasa de aprendizaje estable durante todo el proceso de optimización.

El algoritmo Adam ajusta automáticamente las tasas de aprendizaje para cada parámetro, lo que permite una convergencia más rápida y eficiente en comparación con otros optimizadores. Esta adaptabilidad es especialmente útil en el aprendizaje profundo, donde los modelos pueden contener millones de parámetros.

Su funcionamiento consiste en calcular el primer y el segundo momento de los gradientes para adaptar la tasa de aprendizaje de cada peso en el modelo. Esto permite que los gradientes tomen pasos más grandes o pequeños según la frecuencia y magnitud de su historial, optimizando así el ritmo de aprendizaje de cada peso de la red neuronal.

La fórmula que el optimizador utiliza para la actualización de los pesos es la siguiente:

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Figura 6: Fórmula para calcular la actualización de pesos del optimizador Adam

Donde:

- W_t es el nuevo vector de pesos de la red neuronal en el paso de tiempo t ,
- W_{t-1} es el vector de pesos actual de la red neuronal en el paso de tiempo anterior,

- η es el hiperparámetro que define el tamaño del paso que toma el optimizador también llamado learning rate o tasa de aprendizaje (Adam adapta el tamaño real del paso internamente),
- $\widehat{m}_t$ la estimación del primer momento corregida por sesgo (media móvil exponencial de los gradientes) para determinar la dirección del paso,
- $\widehat{v}_t$ la estimación del segundo momento corregida por sesgo (media móvil exponencial de los gradientes al cuadrado) para normalizar o adaptar la tasa de aprendizaje,
- ϵ la constante de suavizado, generalmente de valor pequeño (10^{-8}) que se agrega al denominador para prevenir la división por cero en el caso de que $\widehat{v}_t$ sea extremadamente pequeño o cero

5.2. Técnicas de Regularización: Early Stopping

Para evitar el sobreajuste al entrenar un modelo con un método iterativo, como el descenso de gradiente se suelen usar técnicas de regularización como el Early Stopping o detención temprana. Estos métodos actualizan el modelo para que se ajuste mejor a los datos de entrenamiento con cada iteración.

Hasta cierto punto, esto mejora el rendimiento del modelo en datos ajenos al conjunto de entrenamiento (por ejemplo, el conjunto de validación). Sin embargo, más allá de ese punto, mejorar el ajuste del modelo a los datos de entrenamiento conlleva un aumento del error de generalización. Las reglas de detención temprana proporcionan una guía sobre el número de iteraciones que se pueden ejecutar antes de que el modelo comience a sobreajustarse.

La regla más extendida sería la de entrenar el modelo monitorizando su rendimiento y guardando sus parámetros al finalizar cada epoch, hasta que apreciemos que el error de validación aumenta de forma sostenida (hay empeoramientos que son debidos a la componente estocástica del algoritmo).

epoch	train_loss	valid_loss	error_rate	time
0	2.300439	1.914173	0.603733	08:21
1	2.001451	1.752479	0.564267	07:59
2	1.847679	1.650396	0.536000	08:00
3	1.657176	1.540547	0.501333	08:00
4	1.529194	1.485027	0.485067	08:00
5	1.435923	1.480115	0.487200	07:59

```

Better model found at epoch 0 with error_rate value: 0.6037333607673645.
Better model found at epoch 1 with error_rate value: 0.5642666816711426.
Better model found at epoch 2 with error_rate value: 0.5360000133514404.
Better model found at epoch 3 with error_rate value: 0.5013333559036255.
Better model found at epoch 4 with error_rate value: 0.48506665229797363.

```

Figura 7: Registro del entrenamiento. En el ejemplo cada vez que la métrica de validación (en este caso, error_rate) disminuye con cada época, deteniéndose cuando el error de validación comience a aumentar de forma sostenida para evitar el sobreajuste.

6. Métricas de Evaluación de Clasificadores

Para evaluar los resultados obtenidos del modelo de clasificación se medirá el nivel de coincidencia de una categoría prevista con la categoría real (Gewarren, s.f.).

6.1. Accuracy, Precision, Recall y F1-Score

Las métricas de evaluación más significativas dependen del modelo y la tarea específicos, el coste de las diferentes clasificaciones erróneas y si el conjunto de datos está equilibrado o desequilibrado. Los verdaderos y falsos positivos y negativos se utilizan para calcular varias de estas métricas que son útiles para evaluar modelos. (Google For Developers, s.f.)

El accuracy o exactitud es la proporción de todas las clasificaciones que fueron correctas, ya sean positivas o negativas, los mejores valores se encuentran cerca de 1 y los peores son cercanos a 0. Se define matemáticamente de la siguiente manera:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Figura 8: Formula para obtener el valor de Accuracy o exactitud

Debido a que incorpora los cuatro resultados de la matriz de confusión (VP, FP, TN y FN), dado un conjunto de datos equilibrado, con cantidades similares de ejemplos en ambas clases, la precisión puede servir como una medida de calidad del modelo de baja granularidad. Por este motivo, a menudo es la métrica de evaluación predeterminada que se usa para modelos genéricos o no especificados que realizan tareas genéricas o no especificadas.

Sin embargo, cuando el conjunto de datos no está equilibrado, o cuando un tipo de error (FN o FP) es más costoso que el otro, que es el caso de la mayoría de las aplicaciones del mundo real, es mejor optimizar para una de las otras métricas.

La Precisión es la proporción de todas las clasificaciones positivas del modelo que realmente son positivas. Se define matemáticamente de la siguiente manera:

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

Figura 9: Formula para obtener el valor de Precision

Cuanto más cerca de 1.00, mejor. Pero exactamente 1,00 indica un problema (normalmente: fuga de etiqueta/destino, sobreajuste o pruebas con datos de entrenamiento). Cuando los datos de prueba están desequilibrados (donde la mayoría de las instancias pertenecen a una de las clases), el conjunto de datos es pequeño o las

puntuaciones se aproximan a 0,00 o 1,00, la precisión no captura realmente la eficacia de un clasificador y necesita comprobar métricas adicionales. (Gewarren, s.f.)

La precisión mejora a medida que disminuyen los falsos positivos, mientras que la recuperación mejora cuando disminuyen los falsos negativos. Sin embargo, aumentar el umbral de clasificación tiende a disminuir la cantidad de falsos positivos y aumentar la cantidad de falsos negativos, mientras que disminuir el umbral tiene los efectos opuestos. Como resultado, la precisión y la recuperación suelen mostrar una relación inversa, en la que mejorar uno de ellos empeora el otro.

El Recall o recuperación es la tasa de verdaderos positivos (TPR), o la proporción de todos los positivos reales que se clasificaron correctamente como positivos. Matemáticamente, la recuperación se define de la siguiente manera:

$$\text{Recall (or TPR)} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

Figura 10: Formula para obtener el valor de Recall

Los falsos negativos son positivos reales que se clasificaron erróneamente como negativos, por lo que aparecen en el denominador. En un conjunto de datos desequilibrado en el que la cantidad de positivos reales es muy baja, la recuperación es una métrica más significativa que la precisión, ya que mide la capacidad del modelo para identificar correctamente todas las instancias positivas. En el caso de aplicaciones como la predicción de enfermedades, es fundamental identificar correctamente los casos positivos. Por lo general, un falso negativo tiene consecuencias más graves que un falso positivo.

La puntuación F1 es la media armónica (un tipo de promedio) de la precisión y la recuperación. Cuanto más cerca de 1.00, mejor. Una puntuación F1 indica cuán preciso es el clasificador y alcanza su mejor valor en 1,00 y la peor puntuación en 0,00. Matemáticamente, se expresa de la siguiente manera:

$$F1 = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}} = \frac{2TP}{2TP + FP + FN}$$

Figura 11: Formula para obtener el valor de f1-score

Esta métrica equilibra la importancia de la precisión y la recuperación, y es preferible a la precisión para los conjuntos de datos con desequilibrio de clases. Cuando la precisión y la recuperación tengan puntuaciones perfectas de 1.0, la F1 también tendrá una puntuación perfecta de 1.0. En términos más generales, cuando la precisión y la recuperación sean similares en valor, F1 estará cerca de su valor. Cuando la precisión y la recuperación estén muy separadas, F1 será similar a la métrica que sea peor.

6.2. Matriz de Confusión

Según Murel, et.al (2025), una matriz de confusión (o matriz de error) es un método de visualización para los resultados del algoritmo clasificador. Más específicamente, es una tabla que desglosa el número de instancias reales de una clase específica frente al número de instancias previstas para esa clase. Las matrices de confusión son una de las varias métricas de evaluación que miden el rendimiento de un modelo de clasificación. Se pueden utilizar para calcular otras métricas de rendimiento del modelo, como la precisión y la coincidencia, entre otras.

		Actual Values	
		Negative (n)	Positive (s)
Predicted Values	Negative (n)	TN	FN
	Positive (s)	FP	TP

Figura 12: Visualización de una matriz de confusión

Se suele representar como una tabla de $N \times N$ que resume la cantidad de predicciones correctas e incorrectas que realizó un modelo de clasificación. La matriz de confusión para un problema de clasificación de varias clases puede ayudar también a identificar patrones de errores.

En su representación, las columnas pueden ser los valores previstos de una clase dada, mientras que las filas representan los valores reales (por ejemplo, datos reales) de una clase determinada, o viceversa.

PREPARACIÓN DE DATOS

La preparación del conjunto de imágenes se llevó a cabo con el objetivo de estandarizar los datos antes de su uso en los modelos de clasificación. En primer lugar, se realizó la carga del dataset y la asignación de etiquetas correspondientes a cada una de las doce categorías de residuos incluidas en el conjunto.

Posteriormente, todas las imágenes fueron redimensionadas a 224×224 píxeles, con el propósito de garantizar un tamaño uniforme como entrada a los modelos. También se aplicó un proceso de normalización de los valores de píxel, lo cual favorece la estabilidad y eficiencia durante el entrenamiento.

El conjunto de datos se dividió en entrenamiento, validación y prueba, permitiendo evaluar el desempeño del modelo en distintas etapas del proceso. Finalmente, se definieron transformaciones básicas, como el ajuste de tamaño y la conversión a tensores, con el fin de asegurar un formato homogéneo antes del modelado.

METODOLOGÍA

El flujo de trabajo del proyecto se dividió en las siguientes etapas:

1. Muestreo (Sample)

Se trabajó con un dataset de 15,150 imágenes de basura doméstica, distribuidas en 12 clases correspondientes a distintos tipos de residuos reciclables y no reciclables. Las imágenes provienen de escenarios reales y semiestructurados, lo que introduce variabilidad en iluminación, fondo y orientación de los objetos. El conjunto completo de datos se dividió en subconjuntos de entrenamiento (70%), validación (15%) y prueba (15%), garantizando una evaluación justa del modelo.

Debido al desbalance entre algunas clases, durante el entrenamiento se implementó un esquema de muestreo ponderado (Weighted Sampling), el cual permitió equilibrar la representación de las clases en cada lote. Este enfoque redujo el sesgo hacia las clases mayoritarias y favoreció un aprendizaje más estable y equilibrado del clasificador.

2. Exploración (Explore)

Se verificó el número de clases, la distribución de las instancias, así como el tamaño y formato de las imágenes. El análisis de la distribución de clases confirmó un alto desbalance entre las 12 categorías. En particular, la clase clothes (ropa) presenta una mayor cantidad de imágenes, mientras que clases como brown-glass (vidrio café), baterías o zapatos cuentan con una representación menor. Este desbalance representa un riesgo para el modelo, ya que puede inducir sesgos hacia las clases mayoritarias.

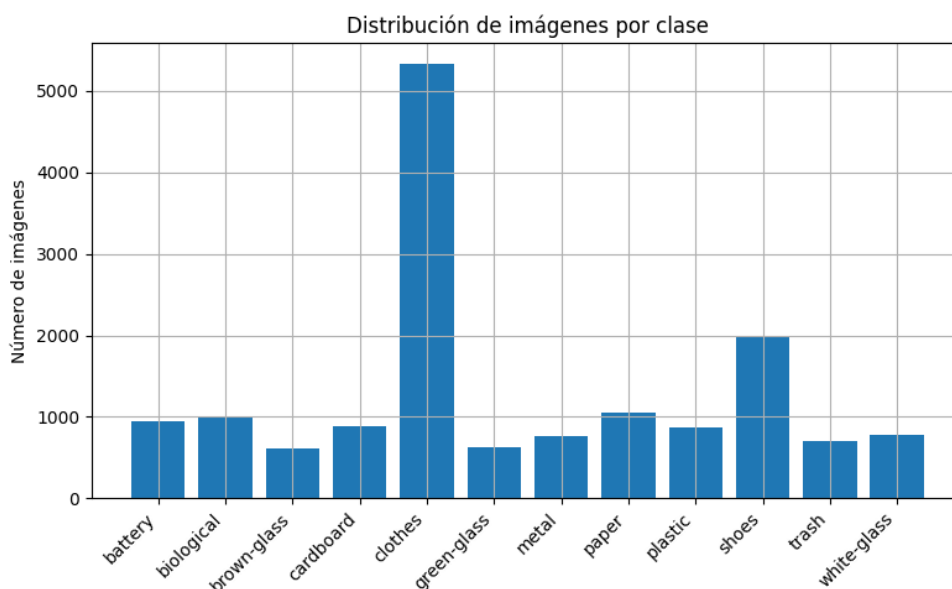


Figura 13: Distribución de imágenes por clase. La clase “clothes” se identificó como la clase mayoritaria.

En cuanto a la calidad y características visuales, se observó que algunas imágenes presentan una apariencia “perfecta”, lo que pone en riesgo que el modelo aprenda patrones asociados al fondo o al contexto de la fotografía en lugar de las características del objeto. Al mismo tiempo, el dataset muestra alta variabilidad visual, con diferencias en iluminación, fondos, resolución y encuadre, lo que simula condiciones reales de uso.

Adicionalmente, se identificaron objetos parcialmente visibles, presencia de ruido visual y similitudes entre ciertos materiales (por ejemplo, distintos tipos de vidrio o plástico), lo que incrementa la dificultad del problema de clasificación.

Como parte del proceso exploratorio, se realizaron visualizaciones de ejemplos representativos por clase y se construyó un DataFrame base para inspeccionar etiquetas, rutas de las imágenes y la distribución de los datos, permitiendo una mejor comprensión del dataset antes de la etapa de preparación y modelado.

Ejemplos de clase: shoes



Ejemplos de clase: trash



Ejemplos de clase: brown-glass



Figura 14: Ejemplos de clase. Cada clase presenta características propias y diferencias visuales notorias.

3. Modificación (Modify)

Como parte del preprocesamiento, todas las imágenes fueron redimensionadas a 224×224 píxeles, tamaño estándar requerido por las arquitecturas convolucionales empleadas. Posteriormente, se realizó la normalización de los valores de píxeles utilizando las medias y desviaciones estándar de ImageNet, lo que permitió una mejor convergencia durante el entrenamiento al mantener la compatibilidad con modelos preentrenados.

Para incrementar la robustez del modelo, se aplicaron técnicas de aumento de datos (Data Augmentation) exclusivamente sobre el conjunto de entrenamiento. Estas transformaciones incluyeron rotaciones aleatorias, volteos horizontales y transformaciones afines leves. Adicionalmente, se empleó la librería Albumentations para aplicar aumentos más avanzados, como ruido Gaussiano, desenfoque de movimiento y ajustes de brillo y contraste.

Finalmente, para abordar el desbalance entre clases durante el entrenamiento, se calcularon pesos por clase (`class_weight`) que fueron incorporados en la función de pérdida de los modelos avanzados, complementando el esquema de muestreo ponderado definido en la etapa de muestreo.

4. Modelado (Model)

En la etapa de modelado se entrenaron y compararon distintos enfoques de clasificación con el objetivo de evaluar el impacto del aprendizaje por transferencia, el manejo del desbalance de clases y la transformación de características en el desempeño del sistema.

4.1 Modelos Exploratorios y de Referencia

Modelo Base: CNN Simple

Como línea base, se implementó una red neuronal convolucional sencilla entrenada desde cero, utilizada para establecer un punto de comparación inicial.

Resultados en el conjunto de prueba:

- Test Loss: 1.2234
- Acerca: 76.42%
- F1-score (weighted): 0.69

Este modelo presentó limitaciones para generalizar ante la variabilidad visual del dataset.

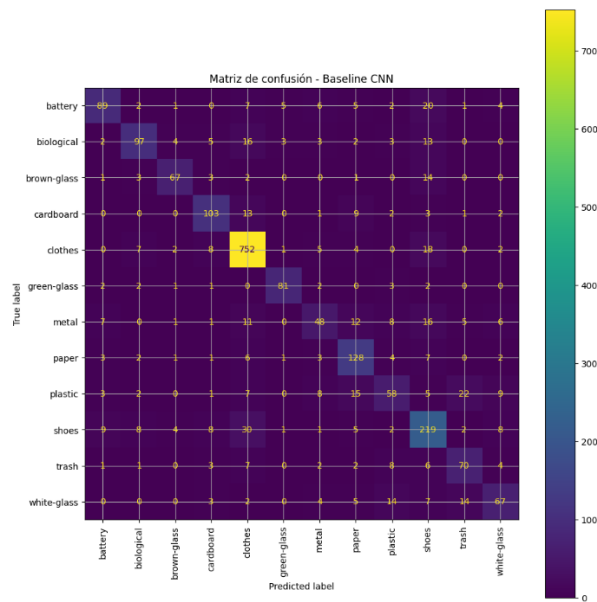


Figura 15: Matriz de confusión – Baseline CNN. Muestra el rendimiento del clasificador: los valores en la diagonal indican las predicciones correctas, siendo "clothes" (ropa) la clase mejor clasificada (752 aciertos).

Transfer Learning: ResNet18

Posteriormente, se evaluó un modelo ResNet18 preentrenado en ImageNet, aplicando fine-tuning en las capas finales.

Resultados en el conjunto de prueba:

- Test Loss: 0.2404
- Accuracy: 92.18%
- F1-score (weighted): 0.89

El uso de aprendizaje por transferencia permitió una mejora sustancial respecto al modelo base, evidenciando la utilidad de las características profundas preentrenadas.

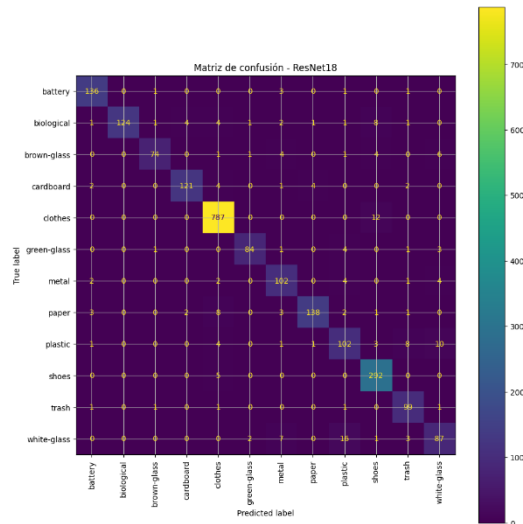


Figura 16: Matriz de confusión – ResNet18. "Clothes" (ropa) fue la clase mejor clasificada (787 aciertos).

ResNet18 con Aumento de Datos y Pesos por Clase

Con el objetivo de mejorar el desempeño en clases minoritarias, se incorporaron técnicas de aumento de datos y pesos por clase durante el entrenamiento.

Resultados en el conjunto de prueba:

- Test Loss: 0.3675
- Accuracy: $\approx 92\%$
- F1-score (weighted): 0.88

Este enfoque permitió estabilizar el proceso de aprendizaje y reducir el sesgo hacia las clases dominantes.

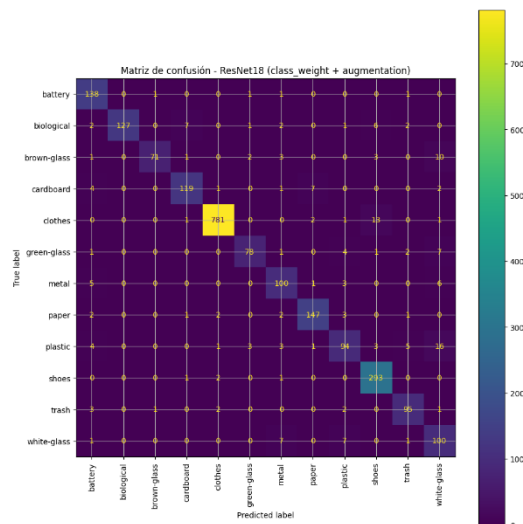


Figura 17: Matriz de confusión – ResNet18 (class_weight + augmentation). "Clothes" (ropa) fue la clase mejor clasificada (781 aciertos).

4.2 Arquitectura Final

ResNet50 + Muestreo Ponderado + LDA

A partir de los resultados obtenidos, se adoptó como modelo final una arquitectura de Transfer Learning basada en ResNet50, organizada en tres etapas principales.

- 1- Arquitectura Base (Feature Extractor): Se utilizó una ResNet50 preentrenada en ImageNet, manteniendo congeladas sus capas convolucionales para extraer representaciones visuales robustas con un costo computacional reducido.
- 2- Clasificador y Entrenamiento: La capa final de clasificación fue reemplazada y entrenada utilizando las características extraídas por la red congelada. Durante el entrenamiento se incorporó muestreo ponderado, con el fin de mitigar el desbalance entre las 12 clases del problema.
- 3- Transformación de Características (LDA): Las características de alta dimensionalidad obtenidas a partir de la capa Global Average Pooling fueron transformadas mediante Análisis Discriminante Lineal (LDA).

Resultados finales del modelo optimizado:

- Accuracy: 95%
- F1-score (weighted): 0.95
- Cohen's Kappa: 0.9343

Estos resultados indican un alto nivel de concordancia entre las predicciones del modelo y las etiquetas reales, consolidando este enfoque como el modelo final del sistema de clasificación de residuos.

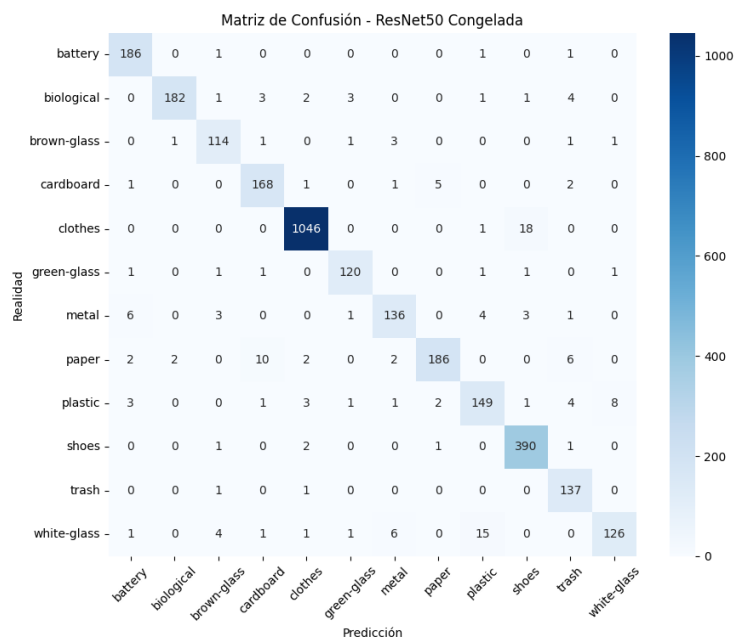


Figura 18: Matriz de confusión – ResNet50 Congelada. Muestra el rendimiento del clasificador: los valores en la diagonal indican las predicciones correctas, siendo "clothes" (ropa) la clase mejor clasificada (1046 aciertos).

5. Evaluación (Assess)

Para la evaluación del modelo se emplearon métricas estándar de clasificación multiclase como Accuracy, Precision, Recall y F1-score, además de la matriz de confusión y el coeficiente de Cohen's Kappa.

El análisis de las curvas Precision–Recall evidenció sobreajuste en los modelos sin reducción de dimensionalidad. En contraste, el modelo que incorpora muestreo ponderado y LDA presentó un comportamiento más estable, indicando una mejor generalización.

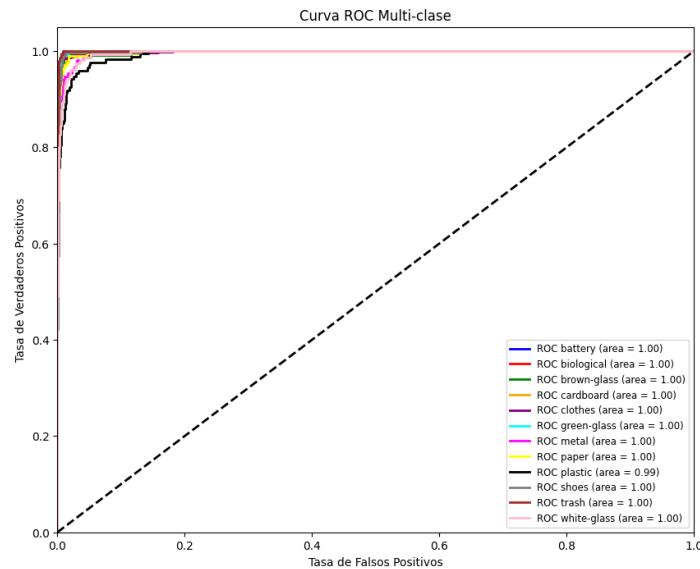


Figura 19: Curva ROC Multiclase ResNet50 Congelado. Muestra el rendimiento casi perfecto del modelo, ya que el Área Bajo la Curva (AUC) es 1.00 para casi todas las 12 clases de residuos.

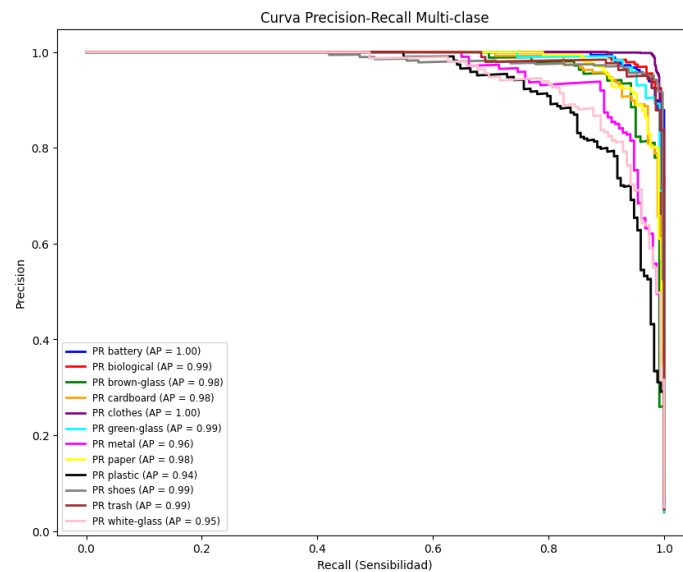


Figura 20: Curva Precision - Recall Multiclase ResNet50 Congelado. Muestra un alto rendimiento, con la Precisión Promedio (AP) en 1.00 para las clases "battery" y "clothes". La clase "plastic" es la de menor rendimiento con un AP de 0.94.

Adicionalmente, el modelo final fue evaluado con imágenes reales capturadas fuera del dataset original, lo que permitió validar su desempeño en escenarios prácticos. En estas pruebas, el modelo con LDA mostró resultados más consistentes.

El modelo final alcanzó un F1-score superior a 0.94 y un Cohen's Kappa mayor a 0.92, confirmando un desempeño robusto incluso en presencia de desbalance y ruido visual.

RESULTADOS

El desempeño de los modelos entrenados evidencia una mejora conforme se incorporaron estrategias más avanzadas de aprendizaje por transferencia, regularización y manejo del desbalance de clases. El modelo base alcanzó una accuracy de 76.42% y un F1-score ponderado de 0.69, mostrando limitaciones claras para generalizar.

Al aplicar aprendizaje por transferencia con arquitecturas ResNet, el desempeño mejoró de manera significativa, alcanzando una accuracy de 92.18% y un F1-score ponderado de 0.89. Sin embargo, las pruebas con imágenes reales tomadas fuera del dataset original evidenciaron que estos modelos presentaban sesgos hacia la clase mayoritaria (*clothes*).

La arquitectura final híbrida (ResNet50 congelada + Muestreo Ponderado + LDA) mostró el mejor desempeño global. Este enfoque alcanzó una accuracy del 95%, F1-score ponderado de 0.95 y un Cohen's Kappa de 0.9343, indicando una concordancia casi perfecta entre predicciones y etiquetas reales. Las curvas ROC y Precision-Recall reflejaron un rendimiento sobresaliente para la mayoría de las clases ($AUC \approx 1.00$), con mejoras notables en estabilidad y generalización frente a modelos puramente neuronales.

DISCUSIÓN

Los resultados confirman que, en un problema de clasificación de residuos con fuerte desbalance entre clases y alta variación visual, los modelos entrenados desde cero no son suficientes. El uso de aprendizaje por transferencia permitió aprovechar modelos ya entrenados, lo que facilitó el aprendizaje y mejoró el desempeño general.

No obstante, las pruebas con imágenes del mundo real mostraron que obtener buenos resultados en el conjunto de prueba no siempre garantiza un buen funcionamiento en situaciones reales. El sesgo hacia la clase mayoritaria (*clothes*) y algunas clasificaciones incorrectas con alta confianza evidenciaron la necesidad de aplicar estrategias adicionales más allá del ajuste básico del modelo.

El muestreo ponderado ayudó a reducir este sesgo durante el entrenamiento, sobre todo cuando se combinó con aumento de datos dinámico, evitando repetir siempre las mismas imágenes de las clases minoritarias. Aun así, la mejora más importante se logró al separar la extracción de características de la etapa final de clasificación, utilizando LDA.

Este enfoque híbrido demostró que las características obtenidas por ResNet50 son altamente informativas y que, al reorganizarlas para separar mejor las clases, se obtuvo un modelo más estable y menos propenso al sobreajuste. En este sentido, el LDA ayudó a controlar la complejidad del modelo y a mejorar su comportamiento general.

CONCLUSIONES

En este proyecto se desarrolló un sistema de clasificación de residuos basado en imágenes, siguiendo un enfoque incremental que permitió evaluar distintas estrategias de modelado. Los resultados evidencian que el uso de aprendizaje por transferencia, junto con técnicas de balanceo y transformación de características, mejora de forma notable el desempeño del clasificador.

El modelo final, basado en ResNet50 congelada con muestreo ponderado y LDA, demostró una mejor capacidad de generalización y una reducción significativa del sesgo hacia las clases mayoritarias. Las pruebas con imágenes reales confirmaron su mayor estabilidad y potencial para aplicaciones prácticas.

En conjunto, este trabajo resalta la importancia de combinar modelos profundos con técnicas clásicas de análisis de datos para obtener sistemas más robustos, especialmente en problemas donde existe desbalance y variabilidad visual.

REFERENCIAS

Clasificación: Exactitud, recuperación, precisión y métricas relacionadas. (s.f., Modificado el 1 de octubre 2025). Google For Developers. https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall?hl=es-419#recall_or_true_positive_rate

Gewarren. (s. f.). *métricas de ML.NET - ML.NET*. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/machine-learning/resources/metrics>

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 770-778). <https://doi.org/10.1109/CVPR.2016.90>

Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1412.6980>

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324. <https://doi.org/10.1109/5.726791>

Murel, J., & Kavlakoglu, E. (2025, 27 noviembre). Matriz de confusión. *IBM*. <https://www.ibm.com/mx-es/think/topics/confusion-matrix>

Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345-1359. <https://doi.org/10.1109/TKDE.2009.191>

Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179-188. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>

Szeliski, R. (2010). *Computer Vision: Algorithms and Applications*. Springer Science & Business Media. <https://doi.org/10.1007/978-1-84882-935-0>

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1905.11946>